



Experiment – 1.1.1

Student Name: Damandeep Singh

UID: 24BDA7O313

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 10-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. Software Required:

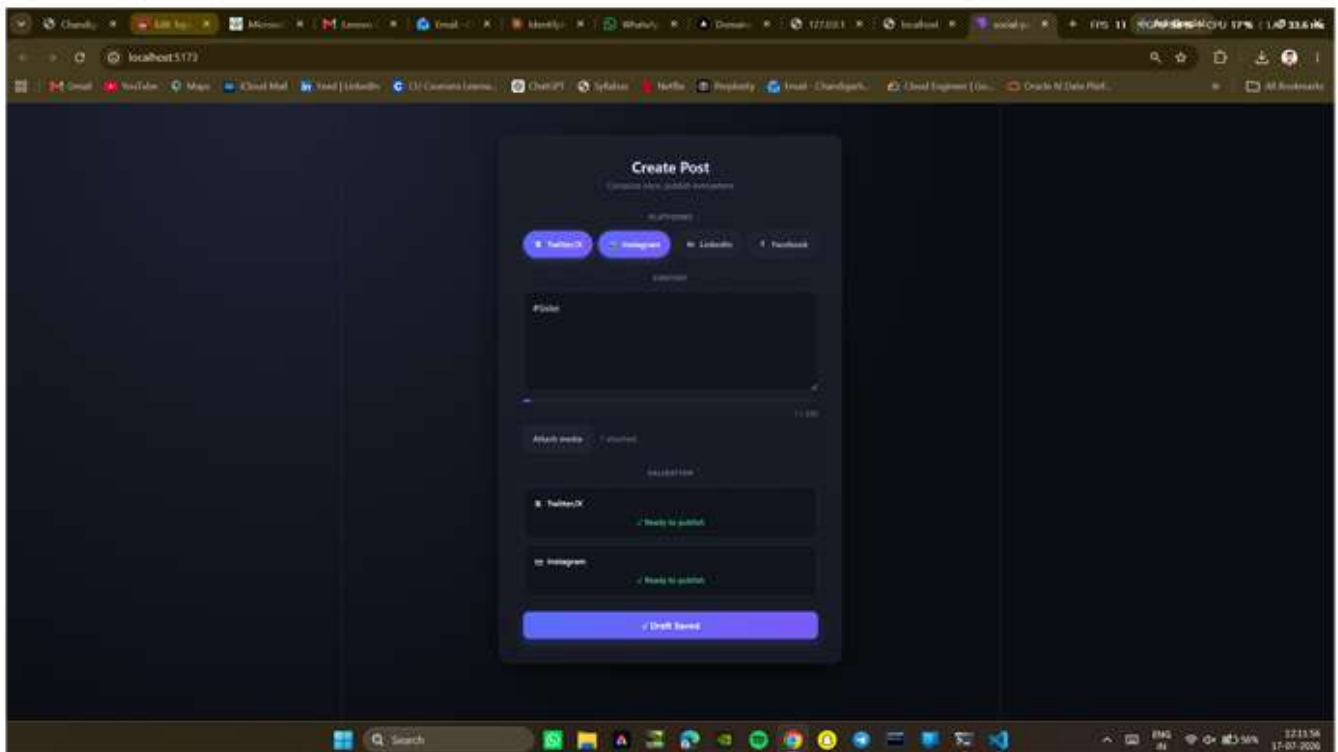
- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

The UI should allow users to input textual content, attach media, and select one or more publishing platforms. Based on platform selection, real-time validation must be enforced to ensure compliance with constraints such as character limits and hashtag rules. Visual feedback mechanisms such as character counters, warnings, and error indicators enhance the user experience. The system should be modular, reusable, and responsive.

4. Theory:

React functional components with Hooks (useState, useMemo, useCallback) allow a UI to derive and recompute values only when their dependencies change. In this experiment, platform metadata (character limits, media limits) is centralized in a Redux store rather than hardcoded per-platform in the component, so the same validation function can be reused across all platforms. Controlled components (textarea bound to state) allow the UI to react instantly to user input, enabling real-time validation feedback.



7. Conclusion:

A single composer screen was successfully built that dynamically adapts its validation and feedback to whichever combination of platforms is selected, without platform-specific branching in the JSX — all rules flow from centralized platform data and one shared validation function.

8. Learning Outcomes:

- Learned to build controlled React components driven entirely by state.
- Understood how to derive real-time validation feedback using `useMemo` without recomputing on unrelated renders.
- Learned to design a UI that is data-driven rather than hardcoded per use case.